

DATA STRUCTURES AND ALGORITHMS

LECTURE 12

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 11...

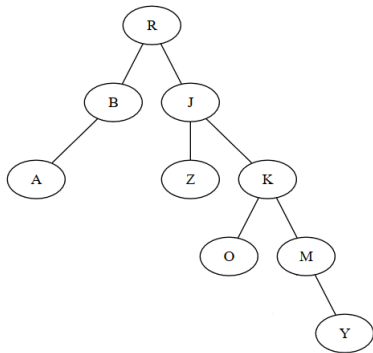
- Cuckoo hashing
- Perfect hashing
- Linked hash table
- Trees
- Binary Tree

- Binary Tree
- Binary Search Tree

Binary trees - recap

- An ordered tree in which each node has at most two children is called *binary tree*.
- In a binary tree we call the children of a node the *left child* and *right child*.
- Even if a node has only one child, we still have to know whether that is the left or the right one.

Binary tree - example



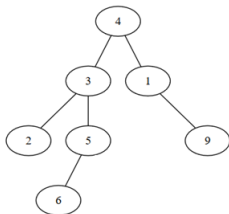
- *A* is the left child of *B*
- *Y* is the right child of *M*

Possible representations

- If we want to implement a binary tree, what representation can we use?
- We have several options:
 - Representation using an array (similar to a binary heap)
 - Linked representation
 - with dynamic allocation
 - on an array

- Representation using an array
 - Store the elements in an array
 - First position from the array is the root of the tree
 - Left child of node from position i is at position $2 * i$, right child is at position $2 * i + 1$.
 - Some special value is needed to denote the place where there is no element.

Possible representations II



Pos	Elem
1	4
2	3
3	1
4	2
5	5
6	-1
7	9
8	-1
9	-1
10	6
11	-1
12	-1
13	-1
...	...

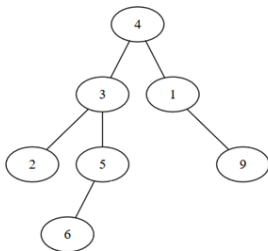
- Disadvantage: depending on the form of the tree, we might waste a lot of space.
- **Obs.** For your lab assignments you are not allowed to use this representation.

- Linked representation with dynamic allocation
 - We have a structure to represent a node, containing the information, the address of the left child and the address of the right child (possibly the address of the parent as well).
 - An empty tree is denoted by the value NIL for the root.
 - We have one node for every element of the tree.

Possible representations IV

- Linked representation on an array
 - Information from the nodes is placed in an array. The *address* of the left and right child is the *index* where the corresponding elements can be found in the array.
 - We can have a separate array for the parent as well.

Possible representations V



Pos	1	2	3	4	5	6	7	8
Info	4	3	2	5	6	1	9	
Left	2	3	-1	5	-1	-1	-1	
Right	6	4	-1	-1	-1	7	-1	
Parent	-1	1	2	2	4	1	6	

- We need to know that the root is at position 1 (could be any position).
- If the array is full, we can allocate a larger one.
- We have to keep a linked list of empty positions to make adding a new node easier.

Possible representations VI

- The linked list of empty positions has to be created when the empty binary tree is created. While a tree is a non-linear data structure, we can still use the left (and/or right) array to create a singly (or doubly) linked list of empty positions.
- Obviously, when we do a resize, the newly created empty positions have to be linked again.

info							
left	2	3	4	5	6	7	8
right							

firstEmpty = 1
root = -1
cap = 8

Binary Tree Traversal

- A node of a (binary) tree is visited when the program control arrives at the node, usually with the purpose of performing some operation on the node (printing it, checking the value from the node, etc.).
- *Traversing* a (binary) tree means visiting all of its nodes.
- For a binary tree there are 4 possible traversals:
 - Preorder
 - Inorder
 - Postorder
 - Level order (breadth first) - the same as in case of a (non-binary) tree

Binary tree representation

- In the following, for the implementation of the traversal algorithms, we are going to use the following representation for a binary tree:

BTNode:

info: TElem

left: ↑ BTNode

right: ↑ BTNode

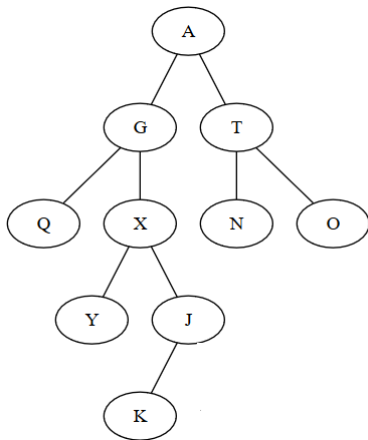
BinaryTree:

root: ↑ BTNode

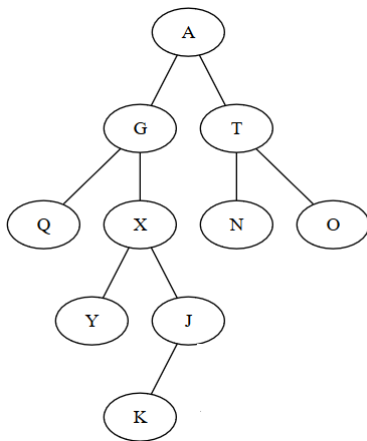
Preorder traversal

- In case of a preorder traversal:
 - Visit the *root* of the tree
 - Traverse the left subtree - if exists
 - Traverse the right subtree - if exists
- When traversing the subtrees (left or right) the same preorder traversal is applied (so, from the left subtree we visit the root first and then traverse the left subtree and then the right subtree).

Preorder traversal example



Preorder traversal example



- Preorder traversal: A, G, Q, X, Y, J, K, T, N, O

Preorder traversal - recursive implementation

- The simplest implementation for preorder traversal is with a recursive algorithm.

subalgorithm preorder_recursive(node) **is:**

//pre: node is a $\uparrow$ BTNode

if node $\neq$ NIL **then**

 @visit [node].info

 preorder_recursive([node].left)

 preorder_recursive([node].right)

end-if

end-subalgorithm

Preorder traversal - recursive implementation

- The *preorder_recursive* subalgorithm receives as parameter a pointer to a node, so we need a wrapper subalgorithm, one that receives a *BinaryTree* and calls the function for the root of the tree.

subalgorithm preorderRec(tree) **is:**

```
//pre: tree is a BinaryTree  
  preorder_recursive(tree.root)
```

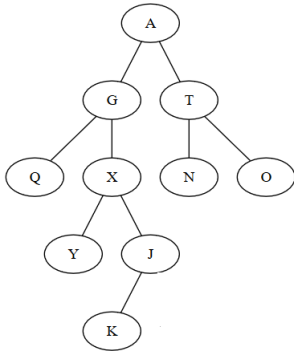
end-subalgorithm

- Assuming that visiting a node takes constant time (print the info from the node, for example), the whole traversal takes $\Theta(n)$ time for a tree with n nodes.

Preorder traversal - non-recursive implementation

- We can implement the preorder traversal algorithm without recursion, using an auxiliary *stack* to store the nodes.
 - We start with an empty stack
 - Push the root of the tree to the stack
 - While the stack is not empty:
 - Pop a node and visit it
 - Push the node's right child to the stack
 - Push the node's left child to the stack

Preorder traversal - non-recursive implementation example



- Stack: A
- Visit A, push children (Stack: T G)
- Visit G, push children (Stack: T X Q)
- Visit Q, push nothing (Stack: T X)
- Visit X, push children (Stack: T J Y)
- Visit Y, push nothing (Stack: T J)
- Visit J, push child (Stack: T K)
- Visit K, push nothing (Stack: T)
- Visit T, push children (Stack: O N)
- Visit N, push nothing (Stack: O)
- Visit O, push nothing (Stack:)
- Stack is empty, traversal is complete

Preorder traversal - non-recursive implementation

```
subalgorithm preorder(tree) is:  
  //pre: tree is a binary tree  
  s: Stack //s is an auxiliary stack  
  if tree.root  $\neq$  NIL then  
    push(s, tree.root)  
  end-if  
  while not isEmpty(s) execute  
    currentNode  $\leftarrow$  pop(s)  
    @visit currentNode  
    if [currentNode].right  $\neq$  NIL then  
      push(s, [currentNode].right)  
    end-if  
    if [currentNode].left  $\neq$  NIL then  
      push(s, [currentNode].left)  
    end-if  
  end-while  
end-subalgorithm
```

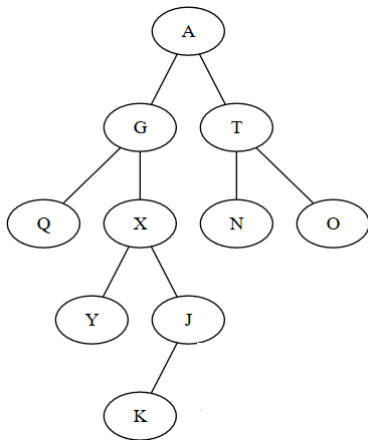
Preorder traversal - non - recursive implementation

- Time complexity of the non-recursive traversal is $\Theta(n)$, and we also need $O(n)$ extra space (the stack)
- Obs: Preorder traversal is exactly the same as *depth first traversal* (you can see it especially in the implementation), with the observation that here we need to be careful to first push the right child to the stack and then the left one (in case of *depth-first traversal* the order in which we pushed the children was not that important).

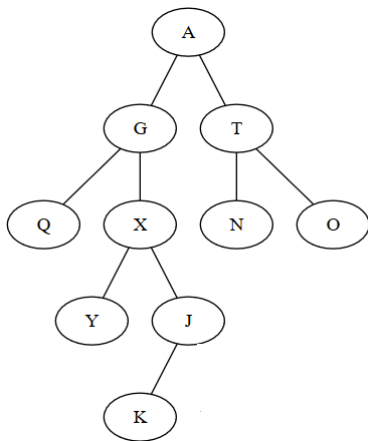
Inorder traversal

- In case of *inorder* traversal:
 - Traverse the left subtree - if exists
 - Visit the *root* of the tree
 - Traverse the right subtree - if exists
- When traversing the subtrees (left or right) the same inorder traversal is applied (so, from the left subtree we traverse the left subtree, then we visit the root and then traverse the right subtree).

Inorder traversal example



Inorder traversal example



- Inorder traversal: Q, G, Y, X, K, J, A, N, T, O

Inorder traversal - recursive implementation

- The simplest implementation for inorder traversal is with a recursive algorithm.

subalgorithm `inorder_recursive(node)` **is:**

//pre: node is a $\uparrow$ BTreeNode

if `node  $\neq$  NIL` **then**

`inorder_recursive([node].left)`

 @visit `[node].info`

`inorder_recursive([node].right)`

end-if

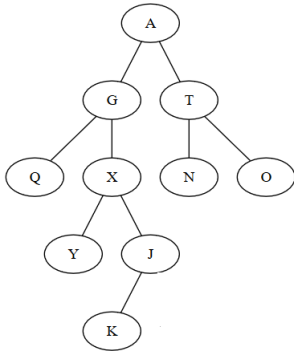
end-subalgorithm

- We need again a wrapper subalgorithm to perform the first call to *inorder_recursive* with the root of the tree as parameter.
- The traversal takes $\Theta(n)$ time for a tree with n nodes.

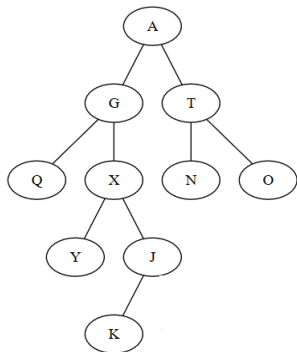
Inorder traversal - non-recursive implementation

- We can implement the inorder traversal algorithm without recursion, using an auxiliary stack to store the nodes.
 - We start with an empty stack and a current node set to the root
 - While current node is not NIL, push it to the stack and set it to its left child
 - While stack not empty
 - Pop a node and visit it
 - Set current node to the right child of the popped node
 - While current node is not NIL, push it to the stack and set it to its left child

Inorder traversal - non-recursive implementation example



Inorder traversal - non-recursive implementation example



- CurrentNode: A (Stack:)
- CurrentNode: NIL (Stack: A G Q)
- Visit Q, currentNode NIL (Stack: A G)
- Visit G, currentNode X (Stack: A)
- CurrentNode: NIL (Stack: A X Y)
- Visit Y, currentNode NIL (Stack: A X)
- Visit X, currentNode J (Stack: A)
- CurrentNode: NIL (Stack: A J K)
- Visit K, currentNode NIL (Stack: A J)
- Visit J, currentNode NIL (Stack: A)
- Visit A, currentNode T (Stack:)
- CurrentNode: NIL (Stack: T N)
- ...

Inorder traversal - non-recursive implementation

```
subalgorithm inorder(tree) is:  
  //pre: tree is a BinaryTree  
  s: Stack //s is an auxiliary stack  
  currentNode  $\leftarrow$  tree.root  
  while currentNode  $\neq$  NIL execute  
    push(s, currentNode)  
    currentNode  $\leftarrow$  [currentNode].left  
  end-while  
  while not isEmpty(s) execute  
    currentNode  $\leftarrow$  pop(s)  
    @visit currentNode  
    currentNode  $\leftarrow$  [currentNode].right  
    while currentNode  $\neq$  NIL execute  
      push(s, currentNode)  
      currentNode  $\leftarrow$  [currentNode].left  
    end-while  
  end-while  
end-subalgorithm
```

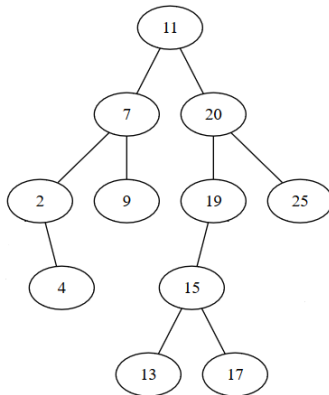
Inorder traversal - non-recursive implementation

- Time complexity $\Theta(n)$, extra space complexity $O(n)$

Binary search trees

- A *Binary Search Tree* is a binary tree that satisfies the following property:
 - if x is a node of the binary search tree then:
 - For every node y from the left subtree of x , the information from y is less than or equal to the information from x
 - For every node y from the right subtree of x , the information from y is greater than the information from x
- Obviously, the relation used to order the nodes can be considered in an abstract way (instead of having " $\leq$ " as in the definition).

Binary Search Tree Example



- If we do an inorder traversal of a binary search tree, we will get the elements in increasing order (according to the relation used).

Binary Search Tree

- The terminology and many properties discussed for binary tree is valid for binary search trees as well:
 - We can have a binary search tree that is full, complete, almost complete, degenerate or balanced
 - The maximum number of nodes in a binary search tree of height N is $2^{N+1} - 1$ - if the tree is complete.
 - The minimum number of nodes in a binary search tree of height N is N - if the tree is degenerate.
 - A binary search tree with N nodes has a height between $\log_2 N$ and N (we will denote the height of the tree by h).

Binary Search Tree

- Binary search trees can be used as representation for sorted containers: sorted maps, sorted multimaps, priority queues, sorted sets, etc.
- In order to implement these containers on a binary search tree, we need to define the following basic operations:
 - search for an element
 - insert an element
 - remove an element
- Other operations that can be implemented for binary search trees (and can be used by the containers): get minimum/maximum element, find the successor/predecessor of an element.

Binary Search Tree - Representation

- We will use a linked representation with dynamic allocation (similar to what we used for binary trees)

BSTNode:

info: TElem

left: $\uparrow$ BSTNode

right: $\uparrow$ BSTNode

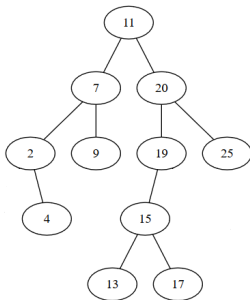
BinarySearchTree:

root: $\uparrow$ BSTNode

- Normally, BST would contain a relation as well. In our examples we will use the $\leq$ relation directly.

Binary Search Tree - search operation

- How can we search for an element in a binary search tree?



- How can we search for element 15? And for element 14?

BST - search operation - recursive implementation

- How can we implement the *search algorithm* recursively?

BST - search operation - recursive implementation

- How can we implement the *search algorithm* recursively?

```
function search_rec (node, elem) is:  
//pre: node is a BSTNode and elem is the TElem we are searching for  
  if node = NIL then  
    search_rec  $\leftarrow$  false  
  else  
    if [node].info = elem then  
      search_rec  $\leftarrow$  true  
    else if [node].info < elem then  
      search_rec  $\leftarrow$  search_rec([node].right, elem)  
    else  
      search_rec  $\leftarrow$  search_rec([node].left, elem)  
    end-if  
  end-function
```


BST - search operation - recursive implementation

- Since the *search* algorithm takes as parameter a node, we need a wrapper function to call it with the root of the tree.

function search (tree, e) **is:**

//pre: tree is a BinarySearchTree, e is the elem we are looking for

search $\leftarrow$ search_rec(tree.root, e)

end-function

BST - search operation - non-recursive implementation

- How can we define the search operation non-recursively?

BST - search operation - non-recursive implementation

- How can we define the search operation non-recursively?

```
function search (tree, elem) is:
```

```
//pre: tree is a BinarySearchTree and elem is the TElem we are searching for
```

```
currentNode  $\leftarrow$  tree.root
```

```
found  $\leftarrow$  false
```

```
while currentNode  $\neq$  NIL and not found execute
```

```
    if [currentNode].info = elem then
```

```
        found  $\leftarrow$  true
```

```
    else if [currentNode].info < elem then
```

```
        currentNode  $\leftarrow$  [currentNode].right
```

```
    else
```

```
        currentNode  $\leftarrow$  [currentNode].left
```

```
    end-if
```

```
end-while
```

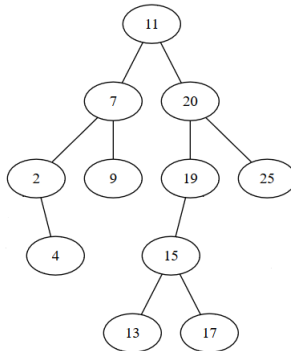
```
search  $\leftarrow$  found
```

```
end-function
```

BST operation complexities

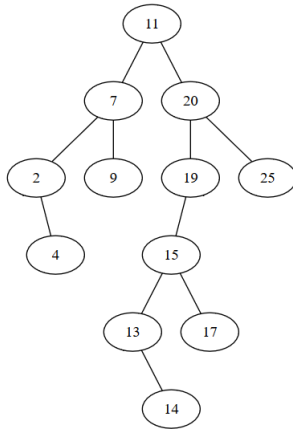
- Most BST operations will follow one single path from the root to a leaf node (or maybe stop somewhere at an internal node, depending on the problem), which means that their complexity depends on the height of the tree, h .
- As discussed, height of the tree can be between $\log_2 n$ and n , so the worst case complexities will in general be $\Theta(n)$ for the operations, which means that total complexity will be in many cases $O(n)$.
- Nevertheless, average complexity is $\Theta(\log_2 n)$ (assuming that the elements were inserted in a random order).
- Regarding the search algorithm, best case complexity is $\Theta(1)$, average case is $\Theta(\log_2 n)$ and worst case is $\Theta(n)$.

BST - insert operation



- How/Where can we insert element 14?

BST - insert operation



BST - insert operation - recursive implementation

- How can we implement the *insert* operation?

BST - insert operation - recursive implementation

- How can we implement the *insert* operation?
- We will start with a function that creates a new node given the information to be stored in it.

function initNode(e) **is:**

//pre: e is a TComp

//post: initNode $\leftarrow$ a node with e as information

allocate(node)

[node].info $\leftarrow$ e

[node].left $\leftarrow$ NIL

[node].right $\leftarrow$ NIL

initNode $\leftarrow$ node

end-function

BST - insert operation - recursive implementation

```
function insert_rec(node, e) is:  
  //pre: node is a BSTNode, e is TComp  
  //post: a node containing e was added in the tree starting from node  
  if node = NIL then  
    node  $\leftarrow$  initNode(e)  
  else if [node].info  $\geq$  e then  
    [node].left  $\leftarrow$  insert_rec([node].left, e)  
  else  
    [node].right  $\leftarrow$  insert_rec([node].right, e)  
  end-if  
  insert_rec  $\leftarrow$  node  
end-function
```

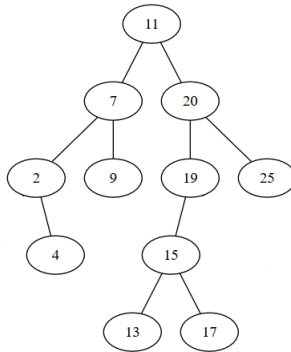
- Complexity:

BST - insert operation - recursive implementation

```
function insert_rec(node, e) is:  
  //pre: node is a BSTNode, e is TComp  
  //post: a node containing e was added in the tree starting from node  
  if node = NIL then  
    node  $\leftarrow$  initNode(e)  
  else if [node].info  $\geq$  e then  
    [node].left  $\leftarrow$  insert_rec([node].left, e)  
  else  
    [node].right  $\leftarrow$  insert_rec([node].right, e)  
  end-if  
  insert_rec  $\leftarrow$  node  
end-function
```

- Complexity: $O(n)$ ($\Theta(n)$ in worst case, but $\Theta(\log_2 n)$ on average)
- Like in case of the *search* operation, we need a wrapper function to call *insert_rec* with the root of the tree.

BST - Finding the minimum element



- How can we find the minimum element of the binary search tree?

BST - Finding the minimum element

```
function minimum(tree) is:  
  //pre: tree is a BinarySearchTree  
  //post: minimum  $\leftarrow$  the minimum value from the tree  
  currentNode  $\leftarrow$  tree.root  
  if currentNode = NIL then  
    @empty tree, no minimum  
  else  
    while [currentNode].left  $\neq$  NIL execute  
      currentNode  $\leftarrow$  [currentNode].left  
    end-while  
    minimum  $\leftarrow$  [currentNode].info  
  end-if  
end-function
```

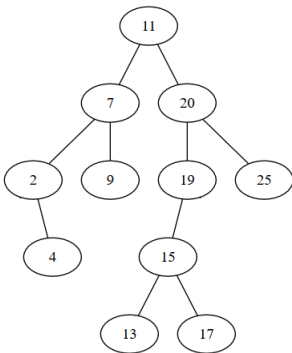
BST - Finding the minimum element

- Complexity of the minimum operation:

BST - Finding the minimum element

- Complexity of the minimum operation: $O(n)$
- We can have an implementation for the minimum, when we want to find the minimum element of a subtree, in this case the parameter to the function would be a node, not a tree.
- We can have an implementation where we return the node containing the minimum element, instead of just the value (depends on what we want to do with the operation)
- Maximum element of the tree can be found similarly.

Finding the parent of a node



- Given a node, how can we find the parent of the node? (assume a representation where the node has no parent field).

Finding the parent of a node

function parent(tree, node) **is:**

//pre: tree is a BinarySearchTree, node is a pointer to a BSTNode, node $\neq$ NIL

//post: returns the parent of node, or NIL if node is the root

$c \leftarrow \text{tree.root}$

if $c = \text{node}$ **then** *//node is the root*

$\text{parent} \leftarrow \text{NIL}$

else

while $c \neq \text{NIL}$ **and** $[c].\text{left} \neq \text{node}$ **and** $[c].\text{right} \neq \text{node}$ **execute**

if $[c].\text{info} \geq [\text{node}].\text{info}$ **then**

$c \leftarrow [c].\text{left}$

else

$c \leftarrow [c].\text{right}$

end-if

end-while

$\text{parent} \leftarrow c$

end-if

end-function

- Complexity:

Finding the parent of a node

function parent(tree, node) **is:**

//pre: tree is a BinarySearchTree, node is a pointer to a BSTNode, node $\neq$ NIL

//post: returns the parent of node, or NIL if node is the root

$c \leftarrow \text{tree.root}$

if $c = \text{node}$ **then** *//node is the root*

$\text{parent} \leftarrow \text{NIL}$

else

while $c \neq \text{NIL}$ **and** $[c].\text{left} \neq \text{node}$ **and** $[c].\text{right} \neq \text{node}$ **execute**

if $[c].\text{info} \geq [\text{node}].\text{info}$ **then**

$c \leftarrow [c].\text{left}$

else

$c \leftarrow [c].\text{right}$

end-if

end-while

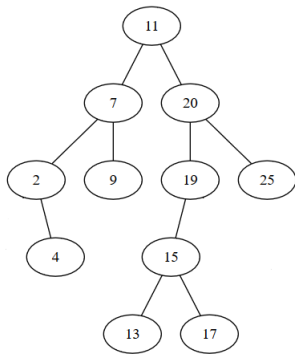
$\text{parent} \leftarrow c$

end-if

end-function

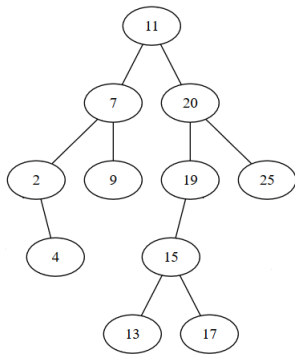
- Complexity: $O(n)$

BST - Finding the successor of a node



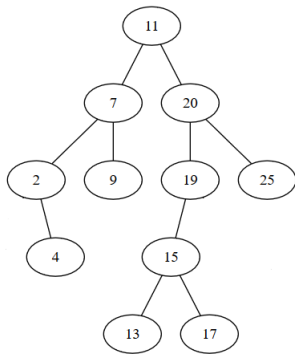
- Given a node, how can we find the node containing the next value (considering the relation used for ordering the elements)?
- How can we find the next after 11?

BST - Finding the successor of a node



- Given a node, how can we find the node containing the next value (considering the relation used for ordering the elements)?
- How can we find the next after 11? After 13?

BST - Finding the successor of a node



- Given a node, how can we find the node containing the next value (considering the relation used for ordering the elements)?
- How can we find the next after 11? After 13? After 17?

BST - Finding the successor of a node

function successor(tree, node) **is:**

//pre: tree is a BinarySearchTree, node is a pointer to a BSTNode, node $\neq$ NIL

//post: returns the node with the next value after the value from node

//or NIL if node is the maximum

if [node].right $\neq$ NIL **then**

 c $\leftarrow$ [node].right

while [c].left $\neq$ NIL **execute**

 c $\leftarrow$ [c].left

end-while

 successor $\leftarrow$ c

else

 p $\leftarrow$ parent(tree, c)

while p $\neq$ NIL **and** [p].left $\neq$ c **execute**

 c $\leftarrow$ p

 p $\leftarrow$ parent(tree, p)

end-while

 successor $\leftarrow$ p

end-if

end-function

BST - Finding the successor of a node

- Complexity of successor:

BST - Finding the successor of a node

- Complexity of successor: depends on parent function:
 - If *parent* is $\Theta(1)$, complexity of successor is $O(n)$
 - If *parent* is $O(n)$, complexity of successor is $O(n^2)$
- What if, instead of receiving a node, successor algorithm receives as parameter a value from a node (assume unique values in the nodes)? How can we find the successor then?
- Similar to successor, we can define a predecessor function as well.

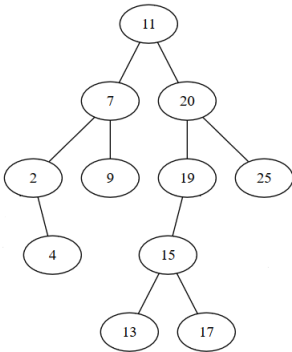
BST - Finding successor of a node

- If we do not have direct access to the parent, finding the successor of a node is $O(n^2)$.
- Can we reduce this complexity?

BST - Finding successor of a node

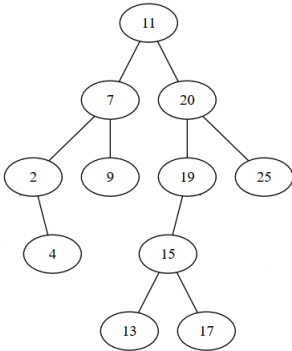
- If we do not have direct access to the parent, finding the successor of a node is $O(n^2)$.
- Can we reduce this complexity?
- $O(n^2)$ is given by the potentially repeated calls for the *parent* function. But we only need the *last* parent, where we went left. We can do one single traversal from the root to our node, and every time we continue left (i.e. current node is greater than the one we are looking for) we memorize that node in a variable (and change the variable when we find a new such node). When the current node is at the one we are looking for, this variable contains its successor.

BST - Remove a node



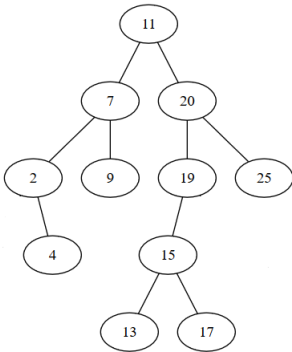
- How can we remove the value 25?

BST - Remove a node



- How can we remove the value 25? And value 2?

BST - Remove a node



- How can we remove the value 25? And value 2? And value 11?

BST - Remove a node

- When we want to remove a value (a node containing the value) from a binary search tree we have three cases:
 - The node to be removed has no descendant
 - Set the corresponding child of the parent to NIL
 - The node to be removed has one descendant
 - Set the corresponding child of the parent to the descendant
 - The node to be removed has two descendants
 - Find the maximum of the left subtree, move it to the node to be deleted, and delete the maximum
- OR**
- Find the minimum of the right subtree, move it to the node to be deleted, and delete the minimum

Binary Search Tree

- Think about it:
 - Can we define a Sorted List on a Binary Search Tree? If not, why not? If yes, how exactly? What would be the most complicated part?

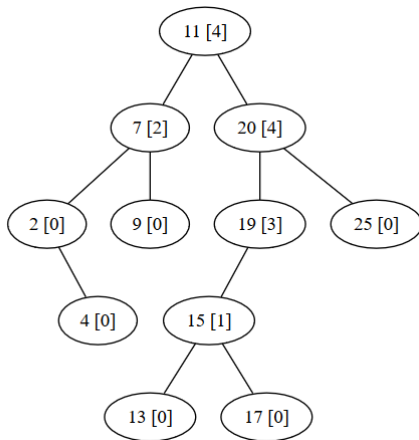
Binary Search Tree

- Think about it:
 - Can we define a Sorted List on a Binary Search Tree? If not, why not? If yes, how exactly? What would be the most complicated part?
 - Lists have positions and operations based on positions. In case of a SortedList we have an operation to remove an element from a position and to return an element from a position (but no insert to position). How can we keep track of positions in a binary search tree?

Binary Search Tree

- Think about it:
 - Can we define a Sorted List on a Binary Search Tree? If not, why not? If yes, how exactly? What would be the most complicated part?
 - Lists have positions and operations based on positions. In case of a SortedList we have an operation to remove an element from a position and to return an element from a position (but no insert to position). How can we keep track of positions in a binary search tree?
 - We can keep in every node, besides the information, the number of nodes in the left subtree. This gives us automatically the "position" of the root in the SortedList. When we have operations that are based on positions, we use these values to decide if we go left or right.

Binary Search Tree



Binary Search Tree with duplicate values

- Starting from an initially empty Binary Search Tree and the relation $\leq$, insert into it, in the given order, the following values: 10, 20, 5, 7, 15, 5, 30, 3, 5, 5, 1, 9, 29, 2.

Binary Search Tree with duplicate values

- Starting from an initially empty Binary Search Tree and the relation $\leq$, insert into it, in the given order, the following values: 10, 20, 5, 7, 15, 5, 30, 3, 5, 5, 1, 9, 29, 2.
- How would you count how many times the value 5 is in the tree?

Binary Search Tree with duplicate values

- Starting from an initially empty Binary Search Tree and the relation $\leq$, insert into it, in the given order, the following values: 10, 20, 5, 7, 15, 5, 30, 3, 5, 5, 1, 9, 29, 2.
- How would you count how many times the value 5 is in the tree?
- Remove 3 (show both options)

Binary Search Tree with duplicate values

- Starting from an initially empty Binary Search Tree and the relation $\leq$, insert into it, in the given order, the following values: 10, 20, 5, 7, 15, 5, 30, 3, 5, 5, 1, 9, 29, 2.
- How would you count how many times the value 5 is in the tree?
- Remove 3 (show both options)
- How would you count now how many times the value 5 is in the tree now?

- Today we have talked about:
 - Binary Trees
 - Binary Search Trees